

Architecture Overview

The 'LLM' class is a core entry point for the TensorRT LLM, providing a simplified 'generate()' API for efficient large language model inference. This abstraction aims to streamline the user experience, as demonstrated with TinyLlama:

```
```python
from tensorrt_llm import LLM

Initialize the LLM with a specified model
llm = LLM(model="TinyLlama/TinyLlama-1.1B-Chat-v1.0")

Generate text using the model
output = llm.generate("Hello, my name is")
```
```

The 'LLM' class automatically manages essential pre and post-processing steps, including tokenization (encoding input prompts into numerical representations) and detokenization (decoding model outputs back into human-readable text).

Internally, the 'LLM' class orchestrates the creation of a dedicated 'PyExecutor(Worker)' process on each rank.

![TensorRT LLM Architecture Overview](../media/TRTLLM_Architecture_Overview.png)

This 'PyExecutor' operates in a continuous background loop, designed for the efficient, asynchronous processing of inference requests.

The 'PyExecutor's functionality is built upon several key components:

- 'Scheduler': Responsible for determining which active requests are ready for execution at each processing step.
- 'KVCacheManager': Manages the allocation, deallocation, and maintenance of the Key-Value (KV) Cache. This is a critical optimization for Transformer models, significantly enhancing performance during autoregressive text generation by storing previously computed attention keys and values.
- 'ModelEngine': Handles the loading and highly efficient execution of the language model on the GPU hardware.
- 'Sampler': Takes the raw outputs (logits) from the ModelEngine and applies appropriate sampling strategies (e.g., greedy, top-k, top-p, beam search) to generate the final output tokens.

During each iteration of its background loop, the 'PyExecutor' performs the following sequence of operations:

- Request Fetching: Retrieves new inference requests from an internal request queue, if available.
- Scheduling: Interacts with the 'Scheduler' to identify and prioritize requests that are ready to be processed in the current step.
- Resource Preparation: Coordinates with the 'KVCacheManager' to ensure that the necessary Key-Value (KV) Cache resources are allocated for the selected requests.
- Model Execution: Invokes the 'ModelEngine' to perform a forward pass on the scheduled requests, predicting the next output tokens.
- Output Handling: Updates the partial outputs for ongoing requests and finalizes the results for any requests that have reached completion, returning them to the user.

Runtime Optimizations

TensorRT LLM enhances inference throughput and reduces latency by integrating a suite of runtime optimizations, including CUDAGraph, [Overlap Scheduler](../features/overlap-scheduler.md), [Speculative decoding](../features/speculative-decoding.md), etc.

CUDAGraph

CUDAGraphs drastically reduce the CPU-side overhead associated with launching GPU kernels, which is particularly impactful in PyTorch-based inference where Python's host-side code can be a bottleneck. By capturing a sequence of CUDA operations as a single graph, the entire sequence can be launched with one API call, minimizing CPU-GPU synchronization and driver overhead.

To maximize the "hit rate" of these cached graphs, TensorRT LLM employs CUDAGraph padding. If an incoming batch's size doesn't match a captured graph, it's padded to the nearest larger, supported size for which a graph exists. While this incurs minor overhead from computing "wasted" tokens, it's often a better trade-off than falling back to slower eager mode execution. This optimization has a significant impact, demonstrating up to a 22% end-to-end throughput increase on certain models and hardware.

Overlap Scheduler

The Overlap Scheduler maximizes GPU utilization by hiding CPU-bound latency behind GPU computation.

The key strategy is to launch the GPU's work for the next step (n+1) immediately, without waiting for the CPU to finish processing the results of the current step (n). This allows the CPU to handle tasks like checking stop criteria or updating responses for one batch while the GPU is already executing the model for the subsequent batch.

This concurrent execution pipeline is illustrated in the 'PyExecutor's logic:

```
```python
Schedule and launch GPU work for the current step (n)
scheduled_batch, _, _ = self._schedule()
batch_outputs = self._forward_step(scheduled_batch, previous_tensors_device)
sample_state = self._sample_async(scheduled_batch, batch_outputs)

While the GPU is busy, process the CPU-bound results from the previous step (n-1)
if self.previous_batch is not None:
 self._process_previous_batch()
```
```

'''

This approach effectively reduces GPU idle time and improves overall hardware occupancy. While it introduces one extra decoding step into the pipeline, the resulting throughput gain is a significant trade-off. For this reason, the Overlap Scheduler is enabled by default in TensorRT LLM.

Visual Generation

For diffusion-based visual generation (image/video), TensorRT-LLM provides a separate 'VisualGen' API and 'DiffusionExecutor' with its own pipeline architecture. See the [Visual Generation](../models/visual-generation.md) feature documentation.

Module-Level Logging

TensorRT-LLM log messages include a fixed-width module tag that identifies which subsystem produced the message:

```
'''text
[TRT-LLM] [I] [runtime ] Loading model weights...
[TRT-LLM] [W] [_torch ] FlashAttention not available, falling back to default
[TRT-LLM] [I] [serve   ] Server listening on port 8000
'''
```

Module Abbreviation Table

Module names longer than 8 characters are abbreviated to fit the fixed-width tag:

| Module Name | Display Tag |
|----------------------|-------------|
| ----- | ----- |
| '_torch' | '_torch ' |
| 'batch_manager' | 'batchmgr ' |
| 'common' | 'common ' |
| 'cutlass_extensions' | 'cutl_ext ' |
| 'deep_ep' | 'deep_ep ' |
| 'deep_gemm' | 'deepgemm ' |
| 'executor' | 'executor ' |
| 'flash_mla' | 'flashmla ' |
| 'kernels' | 'kernels ' |
| 'layers' | 'layers ' |
| 'runtime' | 'runtime ' |
| 'serve' | 'serve ' |
| 'quantization' | 'quantize ' |
| 'scaffolding' | 'scaffold ' |
| 'auto_parallel' | 'autoprl ' |
| 'visual_gen' | 'vis_gen ' |

Per-Module Log Level Filtering

Use 'TLLM_LOG_LEVEL_BY_MODULE' to set different log levels per module, overriding the global 'TLLM_LOG_LEVEL':

```
'''bash
# Format: "level:module1,module2;level:module3"
export TLLM_LOG_LEVEL=warning
export TLLM_LOG_LEVEL_BY_MODULE="debug:_torch,runtime;info:serve"
'''
```

This example sets the global level to 'warning' but enables 'debug' output for '_torch' and 'runtime' modules, and 'info' for 'serve'. Valid levels: 'trace', 'debug', 'verbose', 'info', 'warning', 'error', 'internal_error'.

Multi-Rank Crash Handling

When one rank's executor loop dies in a multi-rank job, the peer ranks have nothing to fail on: they sit in a collective waiting for a rank that will never arrive. Left alone they hold every GPU in the job until an external wall-clock kill.

To avoid that, a rank whose executor loop crashes tears the whole job down ('MPI_Abort', falling back to self-'SIGKILL') after a short grace period. The grace exists so cleaner paths can win the race: if the crash is reported to a client first â\200\224 the caller gets the real traceback â\200\224 the kill stands down, so a symmetric crash still ends in N tracebacks rather than a bare exit 137.

****This is on by default for multi-rank runs.**** Single-rank runs are unaffected; there are no peers to strand.

| Variable | Default | Meaning |
|-----------------------------------|---------|--|
| --- --- --- | | |
| 'TLLM_RANK_CRASH_HARD_KILL_GRACE' | '10' | Seconds to wait before killing the job. '0' kills immediately. Any negative value (e.g. '-1') **disables** the kill entirely â\200\224 peers then fall back to the hang detector. Unparsable and non-finite values ('nan', 'inf') fall back to the default. |

```
'''bash
# Opt out: let the hang detector handle it instead (slower, but no world kill).
export TLLM_RANK_CRASH_HARD_KILL_GRACE=-1
'''
```

Use the escape hatch when you would rather have peer ranks reach their own timeouts â\200\224 for example when attaching a debugger to a surviving rank, or when a harness collects per-rank state that a job-wide abort would destroy.

Set 'TLLM_HANG_DIAGNOSTICS=1' to enable additional PyExecutor diagnostics. It is disabled by default and remains silent unless initialization or execution hangs, then dumps worker stacks and the latest available executor state.

DeepGEMM Barrier Timeout

The DeepGEMM MegaMoE kernels synchronize expert-parallel ranks with in-kernel barriers that self-abort after a fixed wall-clock budget, defaulting to 60 seconds. When the budget is exceeded the kernel prints a diagnostic and trips a device-side assert, which poisons the CUDA context â\200\224 so the first Python-level traceback usually points at whatever unrelated kernel ran next, not at the barrier:

```
```text
DeepGEMM grid sync timeout (60s): sm=114, thread=0, grid_sync_idx=0, ...
DeepGEMM NVLink barrier timeout (60s): rank=5, counter=1381, signal=8, target=16, ...
```
```

Read `signal` against `target`: `signal=8, target=16` means only 8 of the 16 ranks ever arrived at the barrier.

| Variable | Default | Meaning |
|----------------------------------|---------|--|
| 'DG_JIT_BARRIER_TIMEOUT_SECONDS' | '60' | Wall-clock budget, in seconds, for DeepGEMM's in-kernel grid-sync and NVLink barriers. Values '<= 0' are ignored and the built-in default applies. |

```
```bash
Give a large expert-parallel group more headroom before the barrier self-aborts.
export DG_JIT_BARRIER_TIMEOUT_SECONDS=300
```
```

The value is baked into the kernels at JIT compile time and participates in the JIT cache key, so changing it recompiles the affected kernels rather than silently reusing a cached binary.

This is primarily a diagnostic knob: raising it distinguishes a slow-but-healthy rendezvous (all ranks eventually arrive and the run proceeds) from a genuine hang or a rank-count mismatch (the same `signal`/`target` split reappears at the larger budget). Prefer fixing the underlying imbalance over permanently raising the budget, since a longer timeout also delays detection of real hangs.